SargazoRD API

API REST de la Plataforma de Incidencias Yuma

Hito: Diseño, programacion y documentacion de la API REST

Proyecto integrador - Region Yuma, Republica Dominicana

Equipo

Michael Angelo Herrera Made, Melvin Fausto Montero Paniagua, Esbenni Johan Frias Morel, Jhansel Valdez y Joel Villegas Medina

Noviembre 2026

1. Descripcion de la API

SargazoRD es una plataforma para reportar y gestionar la acumulacion de sargazo en las playas de La Romana, Bayahibe y Punta Cana. Esta API REST es el backend del sistema: guarda las playas, los reportes de sargazo (incidencias) y las acciones de limpieza realizadas, y los entrega en formato JSON para que la interfaz web los consuma.

Tecnologias: Node.js y Express. Los datos se guardan en memoria (arreglos), por lo que se reinician al apagar el servidor. **URL base:** http://localhost:3000

2. Recursos

Recurso	Descripcion	Campos
playas	Playas de la Region Yuma	id, nombre, municipio
incidencias	Reportes de sargazo en una playa	id, playaId, severidad, cantidadEstimada, descripcion, estado, fecha
acciones	Intervenciones de limpieza sobre una incidencia (historial)	id, incidenciaId, equipo, descripcion, fecha

Relaciones: una incidencia pertenece a una playa (playaId) y una accion pertenece a una incidencia (incidenciaId).

3. Tabla de endpoints

Metodo	Ruta	Descripcion	Codigos
GET	/api/playas	Listar playas	200
GET	/api/playas/:id	Ver una playa	200, 404
POST	/api/playas	Crear playa	201, 400
GET	/api/incidencias	Listar incidencias (filtro opcional ?severidad=)	200
GET	/api/incidencias/:id	Ver una incidencia	200, 404
POST	/api/incidencias	Reportar sargazo	201, 400
PATCH	/api/incidencias/:id	Cambiar severidad o estado	200, 400, 404
DELETE	/api/incidencias/:id	Eliminar incidencia	204, 404
GET	/api/acciones	Listar historial de acciones	200
POST	/api/acciones	Registrar una accion de limpieza	201, 400

Valores permitidos: severidad = bajo, moderado, critico. cantidadEstimada = baja, media, alta. estado = pendiente, en_proceso, resuelta.

Codigos HTTP usados: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found, 500 Internal Server Error.

4. Ejemplos de solicitudes y respuestas

4.1 GET /api/incidencias?severidad=critico

```
Respuesta (200 OK):
[
  {
    "id": 1,
    "playaId": 1,
    "severidad": "critico",
    "cantidadEstimada": "alta",
    "descripcion": "Orilla cubierta por completo",
    "estado": "pendiente",
    "fecha": "2026-11-01"
  }
]
```

4.2 POST /api/incidencias (reportar sargazo)

```
Solicitud:
{
  "playaId": 1,
  "severidad": "moderado",
  "cantidadEstimada": "media",
  "descripcion": "Sargazo en el muelle"
}

Respuesta (201 Created):
{
  "id": 3,
  "playaId": 1,
  "severidad": "moderado",
  "cantidadEstimada": "media",
  "descripcion": "Sargazo en el muelle",
  "estado": "pendiente",
  "fecha": "2026-11-02"
}
```

4.3 PATCH /api/incidencias/1 (cambiar estado)

```
Solicitud:
{ "estado": "en_proceso" }

Respuesta (200 OK):
{
  "id": 1,
  "playaId": 1,
  "severidad": "critico",
  "cantidadEstimada": "alta",
  "descripcion": "Orilla cubierta por completo",
  "estado": "en_proceso",
  "fecha": "2026-11-01"
}
```

4.4 POST /api/acciones (registrar accion de limpieza)

```
Solicitud:
{
  "incidenciaId": 1,
  "equipo": "Equipo Norte",
  "descripcion": "Limpieza con maquinaria"
}

Respuesta (201 Created):
{
  "id": 2,
  "incidenciaId": 1,
  "equipo": "Equipo Norte",
  "descripcion": "Limpieza con maquinaria",
  "fecha": "2026-11-02"
}
```

4.5 DELETE /api/incidencias/2

```
Respuesta: 204 No Content (sin cuerpo)
```

4.6 Ejemplos de errores

```
POST /api/incidencias con {"playaId": 99, "severidad": "mucho"}
Respuesta (400 Bad Request):
{ "error": "El campo \"playaId\" es obligatorio y debe ser el id de una playa existente" }

PATCH /api/incidencias/1 con {"estado": "xyz"}
Respuesta (400 Bad Request):
{ "error": "\"estado\" debe ser uno de: pendiente, en_proceso, resuelta" }

GET /api/incidencias/999
Respuesta (404 Not Found):
{ "error": "Incidencia no encontrada" }
```

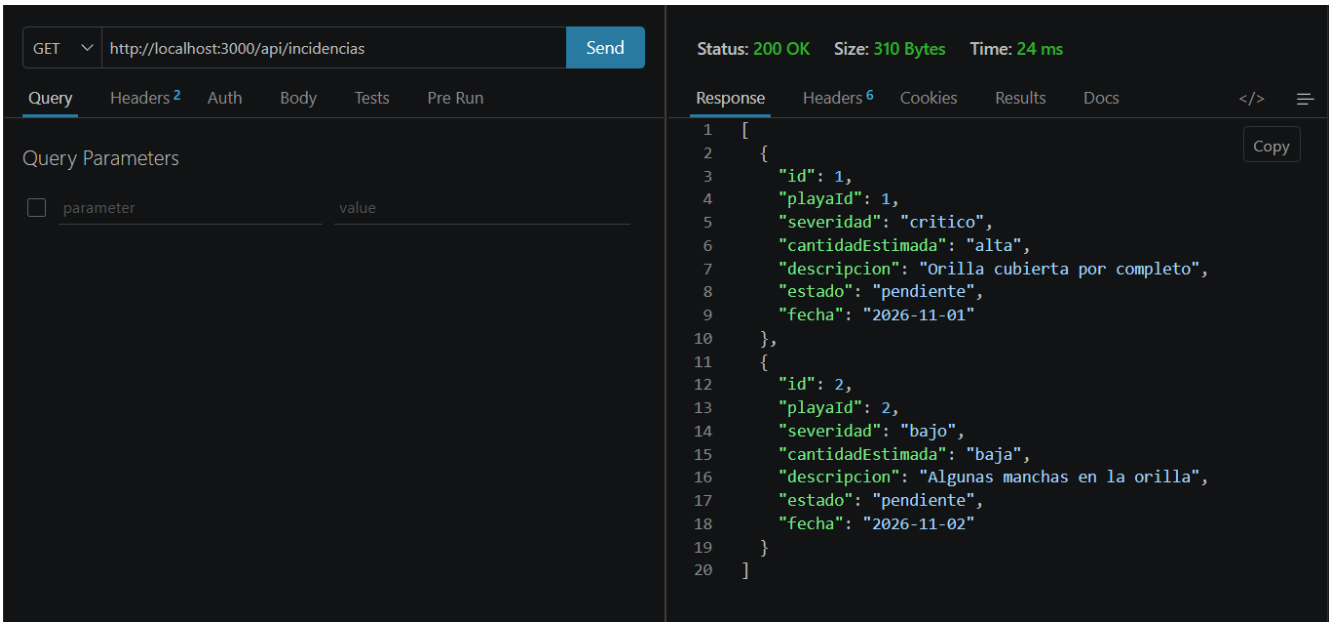
5. Resumen de pruebas realizadas

Todos los endpoints se probaron con Thunder Client (extension de VS Code). En cada prueba se verifico que el codigo de estado y el cuerpo de la respuesta coincidieran con lo esperado.

#	Prueba	Metodo y ruta	Esperado	Resultado
1	Listar playas	GET /api/playas	200	Correcto
2	Ver una playa	GET /api/playas/1	200	Correcto
3	Listar incidencias	GET /api/incidencias	200	Correcto
4	Listar acciones	GET /api/acciones	200	Correcto
5	Crear playa	POST /api/playas	201	Correcto
6	Crear incidencia valida	POST /api/incidencias	201	Correcto
7	Crear incidencia con datos invalidos	POST /api/incidencias	400	Correcto
8	Cambiar estado a en_proceso	PATCH /api/incidencias/1	200	Correcto
9	Estado invalido	PATCH /api/incidencias/1	400	Correcto
10	Incidencia que no existe	GET /api/incidencias/999	404	Correcto
11	Eliminar incidencia	DELETE /api/incidencias/2	204	Correcto
12	Eliminar incidencia ya eliminada	DELETE /api/incidencias/2	404	Correcto
13	Registrar accion	POST /api/acciones	201	Correcto

6. Evidencias de pruebas

A continuacion se muestran las capturas de Thunder Client. Cada captura incluye el metodo, la URL, el cuerpo enviado (en POST y PATCH), el codigo de estado y la respuesta JSON.



Prueba 3: GET /api/incidencias - 200 OK